Homework 2: Make Your Application Learn

Due: 11:59 PM Eastern Time, October 16, 2026 (No Extension Allowed)

1. Overview

In Homework 1, you built the foundation for your own custom research application, mostly for viewing and exploring your data. Now we will take the next step: In this homework, you will add a neural network to your application so that the application can learn from your data and make predictions. The goal is not simply to train a neural network in a notebook. Instead, you will connect the entire pipeline:

Data → Neural Network → Prediction → Interactive Application

By the end of this homework, you should be able to select or provide data through your application and have your neural network produce a prediction that is displayed in the application.

Your specific task will depend on the type of data you chose for HW1. For example:

HW1 Application	Possible HW2 Task
Image/video viewer	Image/video/frame classification
Audio viewer	Audio/sound classification
3D model viewer	3D object/part/point classification
Medical image viewer	Medical image classification
Geographic/map viewer	Spatial classification or regression
Time-series viewer	Time-series classification or forecasting
Language/text viewer	Text classification/regression of sentiment score
Social network viewer	Node classification

Classification is the recommended choice, especially if you are unsure what task to choose. However, if your research data does not naturally fit classification, you may instead implement a regression or another appropriate prediction task. Discuss your proposed task with me or the TA if you are unsure.

2. Detailed Instructions

2.1.Part 1 — Build a Neural Network & a Training Pipeline

Before integrating the model into your application, you will implement and train a small multi-layer perceptron (MLP). The purpose of this part is to connect the concepts we have discussed in class, including perceptrons & multi-layer perceptrons; loss functions & gradients; gradient descent; backpropagation and neural-network training.

You should implement a simple feed-forward neural network consisting of at least one hidden layer, as well as the basic training process. Roughly, the process should look like this:

1. Initialize network parameters
2. Perform forward propagation
3. Calculate the loss

4. Perform backpropagation
5. Calculate parameter gradients
6. Update parameters using gradient descent
7. Repeat 2-6 until the network converges

For this part, you should use PyTorch, TensorFlow, JAX, or other deep learning framework. If you don't know which one to choose, PyTorch is recommended.

The purpose is to make sure that you understand what is happening inside the neural network. You do not need to implement every possible neural-network component from scratch. Instead, you are encouraged to use AI or modify existing codes. However, you must be able to explain your code without the help of AI or other external sources. Also, keep the network small and understandable (don't be a hero, yet).

2.2. Part 2 — Train Your Network

Train your neural network on an appropriate dataset. You may use:

- Your own research data
- A subset of your research data
- A publicly available dataset
- Emulated/synthetically generated data

If your own dataset is difficult to use for this assignment, you may use a simple standard dataset as a starting point. Your goal is to demonstrate that your network can learn a meaningful relationship between inputs and outputs.

Your application or analysis should report at least:

- Training and validation loss (Loss vs Epoch curve)
- Training accuracy or another appropriate performance metric
- Test performance on a set-aside dataset

The exact metrics will depend on your problem.

2.3. Part 3 — Connect the Neural Network to Your HW1 Application

Take the application you built in HW1 and add your trained neural network to it. The user should be able to interact with the application and obtain a prediction. Your application should demonstrate the complete workflow:

User → Select/Provide Data → Your Web App → Neural Network → Prediction → Display Result

The user should not need to manually run a Python script or open a Jupyter notebook to obtain the prediction.

3. AI-Assisted Development

As with HW1, you are encouraged to use LLMs and coding agents. You may use tools such as ChatGPT, Claude, GitHub Copilot, Cursor, or other AI coding assistants. You can use AI to help you:

- Implement the neural network
- Debug your code
- Understand backpropagation
- Prepare your dataset
- Write data-loading code
- Create plots
- Convert your trained model into a form that your application can use
- Connect your model to your web application
- Modify your React application
- Troubleshoot errors

However, you should be able to explain the basic structure of your neural network and training process.

Connecting your model to the web app: This is particularly the area where you can benefit from using an LLM or a coding agent. For AI software services, a typical architecture might look like:

React Web App → HTTP request → FastAPI Backend → PyTorch Model → Prediction → React App

Explore with AI how to implement such a service.

Project organization: You are free to organize your project differently, but a structure such as the following may be useful:

```
my-research-app/
│
├─── frontend/
│    └─── React application
│
├─── backend/
│    ├─── FastAPI application
│    └─── model/
│
├─── data/
│    └─── sample data
│
├─── training/
│    ├─── train.py
│    └─── model.py
│
└─── README.md
```

Do not upload private or sensitive research data to a public GitHub repository. A small sample dataset is sufficient for demonstrating your application.

4. Model Architecture

Do your own research to select model architecture. You are welcome to use/customize the ImageNet backbones we discussed in the course (e.g., VGG, ResNet, GoogLeNet, etc.) or exploit advanced/more recent architectures such as ViT, EfficientNet, message passing, etc.

You are allowed to borrow someone else's implementation and/or use AI to implement your model. In such cases, you must disclose that in your code base (most commonly in the README file). Even if you rely on other people/AI's work to implement your model, in the end, you must do your own work to read the relevant research papers, watch explainer videos, and understand the full details about the model. **You will be asked questions later in the semester, and you are expected to answer them correctly on your own, without looking up the internet or other sources.**

5. Grading

Component	Description	Points
Neural network implementation and training	Submission of a code implementing a neural network model and its training routine, alongside the trained weights. The model should: • Have at least one hidden layer • Perform forward propagation • Calculate a loss • Calculate gradients using backpropagation • Update parameters using gradient descent • Demonstrate that training actually occurs	+30
Reasonable prediction performance	Model can perform a prediction task successfully, where the success is defined as a lower error rate than a shallow model benchmark.	+20
Integration with the HW1 web app	Model can be called from the graphical user interface of the HW1 web app, and the results can be displayed in response.	+30
Late submission	Penalty for late submission	-5 per day
Optional Challenge – Interactive Training	Implement interactive training process that allow the user to modify one or more of: • Learning rate • Number of epochs • Number of hidden units • Number of hidden layers • Activation function • Or other hyperparameters	+10
Optional Challenge – Training Monitor	Implement GUI to display:	+10

	• (Real-time) Training/validation loss • Accuracy and other metrics on train, val, and test data sets • Prediction results	

6. Submission

Submit the following through the Canvas assignment Dropbox:

A. Three-Minute Demo Video

Submit a **maximum 3-minute** video showing a live demonstration of your application. Your video should briefly show: (1) which model you selected and why; (2) how well or poorly it performs (and why); and (3) new features of your web app enabled by the model. If you are submitting for additional credits, the video must also demonstrate them accordingly.

B. GitHub Repository

Submit a link to a GitHub repository containing your source code. Include a `README.md` with:

- A brief description of your application
- The type of data being visualized
- Instructions for running the application
- Major libraries/frameworks used
- A brief description of implemented features
- Whether you implemented annotation
- Whether you implemented a backend/database

7. Notes

At this point, your application has gone from a simple viewer ("I can view my data") to a simple AI-enabled software ("My application can learn and predict something from my data."). As we continue through the semester, you will add more sophisticated models and capabilities.

That said, keep the model simple. Focus on getting the entire pipeline working smoothly (even if it is not super accurate). You will have many opportunities later in the course to make the model more sophisticated.

Disclaimer: ChatGPT was used for proofreading.